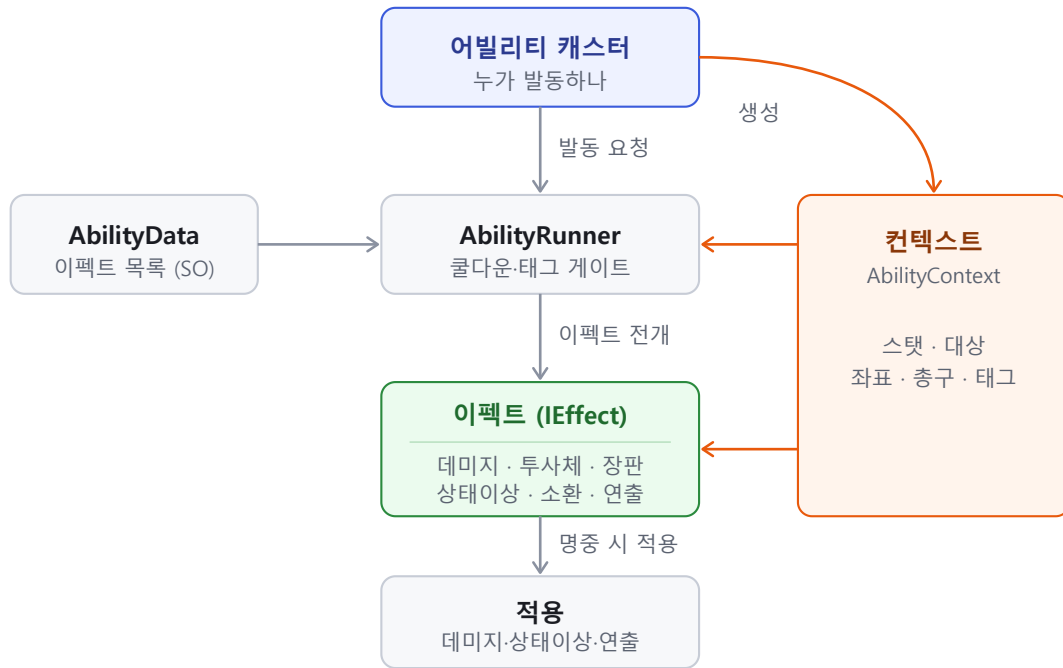


한 번의 발동, 이렇게 흐른다

캐스터 · 컨텍스트 · 러너 · 이펙트가 어떻게 이어지는지, 화살표 하나하나를 쉬운 말로.



□ 캐스터 — 누가 □ 러너·어빌리티·적용 — 뼈대 □ 이펙트 — 무슨 일이 □ 컨텍스트 — 매 단계 동행

그림을 한 문장으로

캐스터가 컨텍스트(이번 상황표)를 만들어 러너에 넘기면, 러너는 어빌리티에 적합한 이펙트들을 그 컨텍스트와 함께 순서대로 실행한다. 세로 화살표(파랑 뼈대)는 "발동이 흐르는 길", 오른쪽 주황 화살표는 "컨텍스트가 매 단계로 실려 들어가는 것"이다.

01 그림은 세 방향으로 읽는다

화살표가 사방으로 뻗어 복잡해 보이지만, 실은 딱 세 갈래다. 이 셋만 구분하면 그림이 저절로 읽힌다.

↓ 세로 · 파란 뼈대 발동이 흐르는 길

위에서 아래로 **캐스터** → **러너** → **이펙트** → **적용**. 이게 "한 번의 발동"이 실제로 지나가는 몸통이다. 누가 발동하든(플레이어·잡몹·보스) 전부 이 한 줄기로 내려온다. 화살표에 붙은 말 — **발동 요청**, **이펙트 전개**, **명중 시 적용** — 이 각 계단에서 무슨 일이 일어나는지다.

→ 왼쪽에서 합류 무엇을 발동할지 (정의)

AbilityData는 "이 스킬은 무슨 이펙트를 어떤 순서로 낸다"가 적힌 **레시피 종이(SO)**다. 스스로 흐르지 않고, 러너가 이걸 **읽어서** 이펙트 목록을 펼친다. 그래서 왼쪽에서 러너로 화살표가 하나 들어온다 — "정의는 옆에서 합류하고, 실행은 뼈대가 한다."

← 오른쪽 레일 매 단계로 실려 가는 상황표

컨텍스트는 흐름의 한 계단이 아니라, 흐름 **전체에 동행하는 상황표**다. 캐스터가 발동하는 순간 딱 한 번만 들어(주황 **생성** 화살표), 그 뒤 러너에도, 각 이펙트에도 **그대로 실려 들어간다** (왼쪽으로 꽃히는 두 주황 화살표). 스탯·대상·좌표·총구가 여기 담겨 있어, 이펙트들이 "어디에·누구에게·얼마의 힘으로"를 안다.

왜 컨텍스트를 "레일"로 그렸나

이펙트가 3개든 10개든, 전부 **똑같은 컨텍스트 하나**를 본다. 첫 총알도 마지막 총알도 같은 조준점·같은 스탯을 쓰는 이유다. 그래서 계단(박스)이 아니라, 흐름 옆을 따라 흐르는 **레일**로 그렸다. 코드로는 `effect.ExecuteAsync(ctx, token)` — 매 이펙트에 같은 `ctx` 가 넘어가는 모습 그대로다.

02 실제로 한 번 따라가 보자

추상적이면 안 와닿으니, “아리스가 평타(라이플 3연사)를 쏜다”를 예로 계단을 하나씩 밟아 본다.

발동 신호가 온다

아리스의 공격 애니메이션이 재생되다가, **총을 쏘는 프레임**에서 애니메이션 이벤트가 “지금!”을 알린다. 발동의 방아쇠는 이렇게 **정확한 프레임**이 당긴다.

캐스터 = 아리스 · **신호** = 발사 프레임의 애니메이션 이벤트

캐스터가 “상황표(컨텍스트)”를 꾸린다

아리스는 지금 이 발동에 필요한 정보를 한 장에 모은다 — 내 **공격력·크리티컬 확률(스탯)**, 지금 겨누는 **대상**, 총알이 나갈 **총구 위치**. 이게 그림 오른쪽 주황 **생성** 화살표다.

컨텍스트 = { 스탯: 아리스 공격력·크리 · 대상: 조준한 적 · 총구: 총구 Transform }

러너가 “지금 써도 되나” 검사한다

캐스터가 **어떤 어빌리티(RifleAttack)**와 **상황표**를 러너에게 넘긴다. 러너는 먼저 게이트를 본다 — 쿨다운이 돌아왔나, “기절” 같은 차단 태그는 없나. 평타는 쿨다운 0이라 바로 통과. 막히면 **여기서 조용히 멈춘다**.

러너 = 아리스의 AbilityRunner · 게이트: 쿨다운 0 ✓, 차단 태그 없음 ✓ → 통과

어빌리티의 이펙트 목록을 펼친다

RifleAttack 레시피에는 “3번 반복 { 총알 1발 스폰 + 총구화염 }”이 적혀 있다. 러너가 이것 읽어 **실행할 이펙트들**로 펼친다. 발수·간격은 코드가 아니라 이 레시피의 **값**일 뿐 — 5연사로 바꾸려면 값만 3→5.

어빌리티 = RifleAttack: Repeat(3) { SpawnProjectiles(1) · SpawnVFX(총구) }

이펙트가 순서대로 실행된다 — 상황표를 들고

각 이펙트는 **같은 컨텍스트**를 받아 자기 일만 한다. 총알 스폰 이펙트는 컨텍스트의 **총구 위치**에서 총알을 만들고, 그 총알에 컨텍스트의 **스탯으로 계산한 데미지 값**을 실어 보낸다. 총구화염 이펙트도 같은 총구 위치에 연출을 낸다. 3번 반복되는 동안 매번 같은 상황표를 본다.

이펙트: 총알 3발 각각 = { 출발: 컨텍스트 총구 · 데미지: 컨텍스트 스탯으로 계산한 스냅샷 }

결과가 적용된다 — 데미지는 “명중” 순간

여기가 초보가 가장 헛갈리는 곳. 데미지는 **발동 순간이 아니라**, 총알이 날아가 **적에게 맞는 순간** 확정된다. 계산은 이미 아리스 스탯으로 해서 총알에 **값 패킷**으로 실어 났으니, 그 사이 아리스가 죽어도(풀에 반 환돼도) 데미지는 안전하게 적용된다.

적용 = 각 총알이 명중하는 순간 TakeDamage · 계산은 발동자, 배달은 총알

03 이 그림이 말하는 핵심 두 가지

복잡한 화살표를 다 걷어내고 남는, 반드시 기억할 두 문장.

① 발동은 "한 줄기"로 흐른다

플레이어든 잡몹이든 보스든, 전부 **캐스터 → 러너 → 이펙트 → 적용**이라는 같은 길을 지난다. 이렇게 길을 하나로 모아 났기 때문에, 쿨다운·태그 게이트·취소 안전 같은 규칙을 러너 한 곳에만 만들면 된다. 새 캐릭터를 추가해도 이 길에 올라타기만 하면 끝 — 발동 로직을 또 짜지 않는다.

② 컨텍스트는 "계단"이 아니라 "동행자"다

캐스터·러너·이펙트는 흐름의 **계단**이지만, 컨텍스트는 그 흐름 옆을 끝까지 따라가는 **상황표**다. 한 번 만들어지면 모든 이펙트가 같은 것을 보므로, "3번째 총알이 갑자기 다른 곳을 조준"하는 일이 없다. 이 하나 덕분에 이펙트들은 서로를 몰라도 되고, 각자 **자기 일(총알·연출·데미지)**만 하면 된다.

비유로 다시

러너는 **요리사**, 어빌리티는 **레시피 종이**, 이펙트는 **조리 단계**, 그리고 컨텍스트는 요리사가 손에 든 **"이번 주문표"**(3번 테이블, 덜 맵게)다. 주문표는 썰 때도 볶을 때도 담을 때도 계속 손에 들려 있다 — 컨텍스트가 매 이펙트에 실려 가는 것과 똑같다.

자주 헷갈리는 지점

"어빌리티가 흐르는 거 아냐?"

아니다. 어빌리티(레시피)는 **가만히 있고**, 러너가 읽는다. 흐르는 건 발동(빠대)과 컨텍스트다.

"컨텍스트는 왜 박스가 아니라 옆의 긴 칸?"

한 계단에서 끝나지 않고 **모든 이펙트에 동행**하기 때문. 그래서 흐름을 따라 흐르는 레일 모양으로 그렸다.

"발동하면 바로 데미지 아냐?"

아니다. 발동은 **총알을 쏘는 데까지**. 데미지는 **명중 순간** 확정. 빗나가면 데미지도 없다.

"연사 5발로 바꾸려면 코드?"

아니다. 레시피의 반복 **값만 3→5**. 이펙트도 러너도 안 건드린다.